

PAJC · PROGRAMMAZIONE AVANZATA IN JAVA E C

L04a

Thread, prima parte

Processi e thread, Runnable e Thread, executor



prof. Massimiliano Redolfi

Università degli Studi di Brescia · Ingegneria Informatica

Oggi

1. **Processi e thread:** che cosa chiede al sistema operativo un `new Thread`
2. **Il primo thread:** `Runnable`, `start`, `sleep`, `join` e l'interruzione
3. **Executor:** pochi thread che vivono a lungo e una coda di compiti

01

Processi e thread



Un programma, un flusso

Il programma come lo abbiamo sempre scritto: una sequenza di istruzioni eseguita da **un unico flusso**, che fa una cosa alla volta.

- quando aspetta (un calcolo lungo, un file, la rete) **aspetta e basta**
- il punto di arrivo: più flussi che lavorano insieme, uno conta, uno legge, uno risponde all'utente, e nessuno tiene fermi gli altri

Prima di scrivere `new Thread(...)` conviene sapere che cosa si sta chiedendo al sistema operativo: il thread di Java è un'astrazione sottile sopra un oggetto che esiste già nel kernel.

Il processo

Un **processo** è un programma in esecuzione: spazio di indirizzamento proprio (codice, dati, heap, stack), tabella dei file aperti, un identificatore, dei diritti.

- la sua proprietà fondamentale è l'**isolamento**: un processo non legge la memoria di un altro, e se va in errore gli altri non se ne accorgono
- il prezzo: crearne uno **costa**, e due processi si parlano solo da fuori (file, pipe, socket)

Il thread

Un **thread** è un flusso di esecuzione *dentro* un processo. Ogni processo ne ha almeno uno (quello di `main`); tutti condividono lo stesso spazio di indirizzamento.

Privato del thread	Condiviso dal processo
stack (variabili locali, chiamate in corso)	heap (tutti gli oggetti)
program counter	codice del programma
registri del processore	variabili globali e statiche
stato (pronto, in esecuzione, bloccato)	file aperti, socket, finestre

La colonna di destra è il motivo per cui i thread **esistono** (costano poco, si scambiano dati leggendo gli stessi oggetti) e la fonte di **ogni loro problema**: se due thread scrivono lo stesso oggetto insieme, nessuno li protegge.

Scheduler e context switch

I thread pronti sono quasi sempre più dei core. Lo **scheduler** decide chi gira dove e per quanto: una **fetta di tempo** di qualche millisecondo, poi si salvano i registri e si carica un altro thread.

- il cambio è il **context switch**: costa poco, non niente
- è **preemptive**: il thread non collabora e non può impedirlo
- **concorrenza**: più flussi in corso nello stesso intervallo, anche su un core solo;
parallelismo: davvero nello stesso istante, su core diversi

Fra due istruzioni qualsiasi può passare un tempo arbitrario, durante il quale gli altri thread fanno quello che vogliono.

Come fanno i sistemi

	Linux	macOS	Windows
un thread è	un <i>task</i> creato con <code>clone</code>	un thread Mach, sotto <code>pthread</code>	l'unità di esecuzione del kernel
chiamata	<code>pthread_create</code> → <code>clone</code>	<code>pthread_create</code>	<code>CreateThread</code>
scheduler	EEVDF: quota equa di tempo	priorità e classi di servizio	32 priorità, boost agli interattivi
stack per thread	8 MB	512 KB	1 MB

- tutti schedulano **thread**, non processi: il processo è un contenitore di risorse

Lo stesso programma, nei due mondi

JAVA

```
Thread t = new Thread(() ->
    System.out.println("ciao da un altro thread");
t.start();
System.out.println("ciao dal thread principale");
t.join();
```

Due metodi: `start` e `join`

C, CON PTHREAD

```
void *lavoro(void *arg) {
    printf("ciao da un altro thread\n");
    return NULL;
}

int main(void) {
    pthread_t t;
    pthread_create(&t, NULL, lavoro, NULL);
    printf("ciao dal thread principale\n");
    pthread_join(t, NULL);
    return 0;
}
```

Due chiamate: `pthread_create` e `pthread_join`

L'astrazione di Java

Un thread è un oggetto `java.lang.Thread`. A `start()` la JVM chiede al sistema un thread nativo e vi lega l'oggetto: stesso ciclo di vita, stesso stack (1 MB, `-Xss`), stesso scheduler. È il **thread di piattaforma**.

- il programma **non è mai solo**: un `main` che stampa una riga gira in una JVM con una ventina di thread (garbage collector, JIT, `Signal Dispatcher`, ...)
- si vedono nella vista *Debug* di Eclipse o con `jstack`, che stampa lo stack di ogni thread: il modo più diretto per capire un programma bloccato
- la JVM termina quando finisce l'ultimo thread **non daemon**; `setDaemon(true)` prima di `start()` marca un thread di servizio

Gli stati di un thread

Thread.State	Che cosa significa	Per il SO
NEW	creato, <code>start()</code> non ancora chiamato	non esiste
RUNNABLE	in esecuzione, pronto o in attesa di I/O	in esecuzione / pronto / bloccato
BLOCKED	aspetta di entrare in un <code>synchronized</code>	bloccato
WAITING	<code>wait()</code> , <code>join()</code> , <code>park()</code> senza scadenza	bloccato
TIMED_WAITING	<code>sleep(ms)</code> , <code>wait(ms)</code> , <code>join(ms)</code>	bloccato
TERMINATED	<code>run()</code> è finito, bene o male	non esiste più

Un thread bloccato **non consuma processore**: cento thread in attesa di rete costano quasi nulla, dieci thread che calcolano saturano dieci core.

02

Il primo thread



Compito e lavoratore

Java separa **che cosa** va eseguito da **chi** lo esegue: il compito è un `Runnable`, il lavoratore un `Thread`.

```
Runnable task = () -> {  
    String nome = Thread.currentThread().getName();  
    System.out.println("Ciao, sono " + nome);  
};  
  
task.run();                                // 1: lo eseguo io  
  
Thread thread = new Thread(task);  
thread.start();                            // 2: lo esegue un altro thread  
  
System.out.println("Fatto!");              // 3
```

start **ritorna subito**

```
Ciao, sono main  
Fatto!  
Ciao, sono Thread-0
```

- `task.run()` è una chiamata di metodo come le altre: la esegue `main`
- `start()` chiede un nuovo thread e **ritorna senza aspettare**; `main` stampa «Fatto!»
- intanto `Thread-0` viene schedulato, esegue `run()` e stampa a sua volta
- l'ordine delle ultime due righe **non è garantito**: due thread non hanno un ordine, a meno che il programma non lo imponga

`thread.run()` al posto di `thread.start()` compila, gira e non crea nessun thread: `run` è un

Un compito con dei dati

```
public class MyTask implements Runnable {
    private final String nome; private final int nStep, delay;
    public MyTask(String nome, int nStep, int delay) {
        this.nome = nome; this.nStep = nStep; this.delay = delay;
    }
    public void run() {
        for (int i = 0; i < nStep; i++) {
            System.out.printf("%s, passo %2d [%s]\n",
                               nome, i, Thread.currentThread().getName());
            try {
                Thread.sleep(delay);
            } catch (InterruptedException e) {
                return; // ci hanno chiesto di smettere
            }
        }
    }
}
```

Due compiti, due thread

```
new Thread(new MyTask("A", 5, 30)).start();  
new Thread(new MyTask("B", 5, 20)).start();  
System.out.println("main ha finito");
```

```
main ha finito  
A, passo 0 [Thread-0]  
B, passo 0 [Thread-1]  
B, passo 1 [Thread-1]  
A, passo 1 [Thread-0]  
B, passo 2 [Thread-1]  
...
```

- `main` finisce subito, ma il programma no: la JVM resta viva finché `A` e `B` non hanno completato `run`

Aspettare: `sleep` e `join`

```
Thread a = new Thread(new MyTask("A", 5, 30));  
a.start();  
a.join(); // main aspetta A  
System.out.println("A ha finito, ora tocca a main");
```

- `Thread.sleep(ms)` : il thread corrente va in `TIMED_WAITING` per **almeno** `ms` millisecondi; non consuma processore, il risveglio dipende dallo scheduler
- `t.join()` : il thread corrente aspetta che `t` termini. È il modo per **imporre un ordine**, quando serve

Entrambi dichiarano `InterruptedException`, un'eccezione checked. Non è pedanteria: è l'intero meccanismo con cui in Java si chiede a un thread di smettere.

Fermare un thread: l'interruzione

Un thread **non si uccide** dall'esterno (`stop()` è deprecato da decenni e oggi inoperante). Si può solo chiedergli di fermarsi, e lui deve collaborare.

- `t.interrupt()` alza una **bandiera** nel thread `t`
- se `t` sta **calcolando**, la controlla a ogni giro con `Thread.currentThread().isInterrupted()`
- se `t` è in **attesa** (`sleep` , `join` , `wait` , una coda bloccante), l'attesa si interrompe con `InterruptedException` e la bandiera si abbassa

```
public void run() {  
    while (!Thread.currentThread().isInterrupted()) {  
        faiUnPassoDiCalcolo();  
    }  
}
```

Il `catch` vuoto

`catch (InterruptedException e) { }`, quello che Eclipse genera, **ingoia la richiesta**: la bandiera è già stata abbassata dall'eccezione, il ciclo riprende e il thread non si fermerà mai.

Nel `catch` si fa una di due cose: si esce (`return`, come in `MyTask`), oppure si **rialza la bandiera** e si lascia decidere a chi sta sopra. Così fa `pausa`, che useremo negli esempi:

```
static void pausa(long ms) {  
    try {  
        Thread.sleep(ms);  
    } catch (InterruptedException e) {  
        Thread.currentThread().interrupt();    // rialza la bandiera e ritorna  
    }  
}
```

03

Executor



Perché non un thread per ogni compito

Con `new Thread(task).start()` ogni compito costa un thread del sistema: uno stack riservato, una chiamata al kernel, un context switch in più.

- per dieci compiti è irrilevante; per **diecimila** è un problema di memoria e di tempo
- per un flusso continuo di piccoli compiti (le richieste di un server, gli eventi di un'interfaccia) è il **modo sbagliato di pensare**
- il modello giusto: **pochi thread che vivono a lungo** e una coda di compiti da cui pescano

In Java si chiama **executor**, sta in `java.util.concurrent` e separa definitivamente il *che cosa* dal *chi*.

ExecutorService

UN THREAD

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();  
executor.submit(new MyTask("A", 3, 30));  
executor.submit(new MyTask("B", 3, 20));  
executor.submit(new MyTask("C", 3, 10));  
System.out.println("Fatto!");  
executor.shutdown();
```

Fatto!

```
A, passo 0 [pool-1-thread-1]  
A, passo 1 [pool-1-thread-1]  
A, passo 2 [pool-1-thread-1]  
B, passo 0 [pool-1-thread-1]  
...
```

QUANTI SERVONO

```
ExecutorService executor =  
    Executors.newCachedThreadPool();  
executor.submit(new MyTask("A", 3, 30));  
executor.submit(new MyTask("B", 3, 20));  
executor.submit(new MyTask("C", 3, 10));  
System.out.println("Fatto!");  
executor.shutdown();
```

Fatto!

```
A, passo 0 [pool-1-thread-1]  
B, passo 0 [pool-1-thread-2]  
C, passo 0 [pool-1-thread-3]  
C, passo 1 [pool-1-thread-3]  
...
```

Le factory di Executors

Factory

Che cosa fa

`newSingleThreadExecutor()`

un thread, compiti in sequenza: serializza senza bloccare chi li manda

`newFixedThreadPool(n)`

`n` thread fissi e una coda illimitata: il caso tipico per il calcolo

`newCachedThreadPool()`

crea thread quando servono, li riusa, li elimina dopo un minuto: molti compiti brevi che aspettano

`newScheduledThreadPool(n)`

come il pool fisso, ma con compiti ritardati o periodici

`newWorkStealingPool()`

un `ForkJoinPool` con un thread per core: lo stesso degli stream paralleli

execute • submit

```
executor.execute(task);           // esegue e basta  
Future<?> f = executor.submit(task); // esegue e restituisce un Future
```

- con `execute`, un'eccezione non catturata nel compito termina il thread del pool (rimpiazzato) e viene **stampata** dal gestore di default
- con `submit`, l'eccezione **non viene stampata**: resta conservata nel `Future`, e chi non lo interroga non la vedrà mai

Trappola classica: un compito che fallisce in silenzio. Se usate `submit`, prima o poi chiamate `get()` sul risultato.

Un executor va spento

```
ExecutorService executor = Executors.newSingleThreadExecutor();  
executor.submit(() -> System.out.println("Ciao da "  
    + Thread.currentThread().getName()));
```

```
Ciao da pool-1-thread-1
```

```
-
```

```
<- e il programma non termina
```

I thread di un pool **non sono daemon** e aspettano altri compiti, per sempre: la JVM non esce.

Spegnere un executor

- `shutdown()` : niente nuovi compiti, quelli in coda vengono completati
- `shutdownNow()` : svuota la coda e **interrompe** i compiti in corso
- `awaitTermination(t, unit)` : aspetta che tutto finisca, al massimo `t`

```
executor.shutdown();           // 1. niente nuovi compiti
try {
    if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
        executor.shutdownNow(); // 2. troppo lenti: interrompi
    }
} catch (InterruptedException e) {
    executor.shutdownNow();      // 3. interrotti: interrompi tutti
    Thread.currentThread().interrupt();
}
```

Oppure: try con risorse

ExecutorService è AutoCloseable **JAVA 19**: `close()` fa `shutdown` seguito da un'attesa illimitata.

```
try (ExecutorService ex = Executors.newFixedThreadPool(4)) {  
    ex.submit(new MyTask("A", 3, 30));  
    ex.submit(new MyTask("B", 3, 20));  
}                                     // qui tutto e' finito
```

Se l'executor nasce e muore nello stesso metodo, è la forma più breve e corretta: niente `shutdown` dimenticati, niente programmi che non terminano.

Dentro un pool

Una coda più un gruppo di thread che eseguono tutti lo stesso ciclo: estrai un compito, eseguillo, ricomincia. Chi fa `submit` non aspetta nessuno; chi esegue non sa chi ha mandato il compito.



Quanti thread?

Dipende da che cosa fanno i compiti.

- **calcolano** (CPU-bound): più thread dei core non servono, si alternano sugli stessi core pagando i context switch; `Runtime.getRuntime().availableProcessors()` dà il numero giusto
- **aspettano** (I/O-bound: rete, disco, un altro servizio): per la maggior parte del tempo sono bloccati e non occupano un core; possono essere molti di più, il limite è la memoria degli stack

Il pool con `n` thread fissi è la scelta di default per il calcolo; il pool *cached* per i compiti brevi che aspettano. Per l'I/O massiccio Java 21 offre un'alternativa migliore di entrambi: i thread virtuali, nella terza parte.

In pratica

Quando	Che cosa
un compito che agisce (stampa, aggiorna, scrive)	<code>Runnable</code>
un thread solo, che vive quanto il programma	<code>new Thread(task).start()</code>
molti compiti, o un flusso continuo	un <code>ExecutorService</code> , chiuso con <code>shutdown</code> o <code>close</code>
calcolo puro	pool fisso con un thread per core
compiti brevi che aspettano	pool <i>cached</i>

Errori frequenti

- `run()` **al posto di** `start()` : nessun thread, programma sequenziale, nessun errore
- **assumere un ordine** fra due thread: non c'è, finché non lo impone un `join`
- `catch (InterruptedException e) { }` **vuoto**: il thread non si fermerà mai
- **executor mai spento**: il programma non termina
- `submit` **senza mai leggere il** `Future` : le eccezioni spariscono
- **più thread dei core** per un calcolo puro: context switch, non velocità

Compiti che agiscono, compiti che calcolano

Finora ogni compito **agisce**: stampa, aspetta, scrive. Un `Runnable` non restituisce niente.

- un compito che **calcola** un risultato ha bisogno di restituirlo a chi lo ha mandato:
`Callable` e `Future`
- con più compiti che calcolano insieme si misura **quanto si guadagna** davvero: i numeri primi fino a 10^8
- e poi il caso che finora abbiamo evitato: due thread che toccano **lo stesso dato**

Prossima lezione: `Callable` e `Future`, i primi fino a 10^8 , i conflitti e `synchronized`.